

VITyarthi – Build Your Own Project

HOTEL MANAGEMENT SYSTEM

Project Report

Name: Nitish Kumar

Registration Number: 26BAI10542

Faculty: Velmurugan L

Technology: Python

Storage: JSON File Handling

Project Type: Menu-Driven Management System

Submitted as part of the Build Your Own Project evaluation

VIT Bhopal University

1. Introduction

The Hotel Management System is a Python-based application designed to simplify common hotel operations. It provides a menu-driven interface for managing rooms, guests, bookings, check-in/check-out, billing and basic reports.

2. Problem Statement

Manual handling of hotel records can result in duplicated information, difficulty tracking room availability, and errors during billing. The proposed system organizes these activities in one application.

3. Objectives

- Maintain room information.
- Register and manage guests.
- Process room bookings.
- Manage check-in and check-out.
- Calculate bills.
- Produce basic hotel reports.
- Demonstrate Python programming concepts in a real-world application.

4. Functional Requirements

Module 1: Room Management

- Display all rooms.
- Show room type, price and availability.
- Update room status.

Module 2: Guest Management

- Register a guest.
- Generate a unique guest ID.
- Display guest records.
- Search for a guest.

Module 3: Booking Management

- Book an available room.
- Check-in a guest.
- Check-out a guest.
- Track booking status.

Module 4: Billing

- Calculate room charges.
- Accept food/service charges.
- Calculate 5% tax.
- Display total bill.

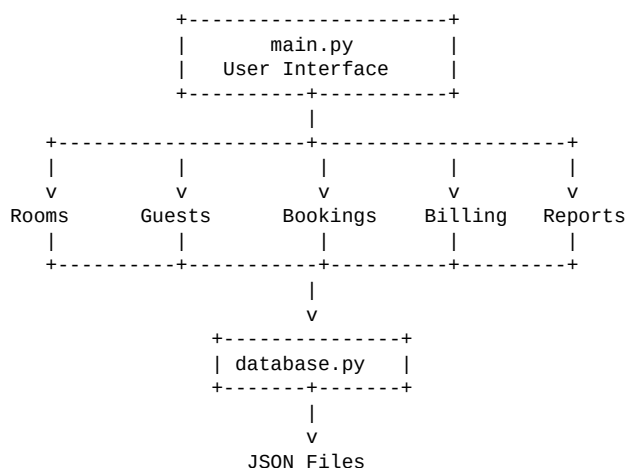
Module 5: Reporting

- Count rooms.
- Show available and occupied rooms.
- Count guests and bookings.
- Calculate room revenue.

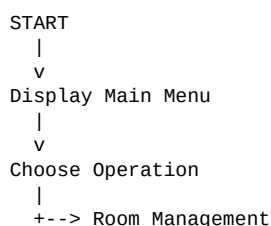
5. Non-Functional Requirements

- **Usability:** The menu-driven interface should be easy for a beginner to use.
- **Reliability:** Invalid input should not terminate the application.
- **Maintainability:** Different operations are separated into Python modules.
- **Error Handling:** Missing files, invalid numbers, unavailable rooms and unknown IDs are handled with appropriate messages.
- **Resource Efficiency:** The system uses lightweight JSON files for local storage.
- **Scalability:** Additional modules such as food ordering or employee management can be added later.

6. System Architecture



7. Process Flow / Workflow



```

|
+--> Guest Registration
|
+--> Room Booking
|   |
|   | v
|   | Check Availability
|   | |
|   | | v
|   | | Booking
|   | |
+--> Check-In
|
+--> Billing
|
+--> Check-Out
|
+--> Reports
|
v
Exit

```

8. Use Case Diagram

HOTEL MANAGEMENT SYSTEM

```

Receptionist / Admin
|
+-----> Manage Rooms
|
+-----> Register Guest
|
+-----> Book Room
|
+-----> Check-In
|
+-----> Check-Out
|
+-----> Generate Bill
|
+-----> View Reports

```

9. Class / Component Diagram

```

RoomManager
- rooms
+ display_rooms()
+ find_room()
+ available_rooms()
+ set_status()

GuestManager
- guests
+ add_guest()
+ find_guest()
+ display_guests()

BookingManager
- bookings
+ book_room()
+ display_bookings()
+ find_booking()
+ check_in()
+ check_out()

BillingManager
+ generate_bill()

Reports
+ show_report()

Database Functions
+ load_data()
+ save_data()

```

10. Sequence Diagram

```
User -> Main: Select "Book Room"
Main -> GuestManager: Find Guest
GuestManager -> Main: Guest Details
Main -> RoomManager: Check Room
RoomManager -> Main: Availability
Main -> BookingManager: Create Booking
BookingManager -> Database: Save Booking
BookingManager -> RoomManager: Update Status
Main -> User: Booking Confirmation
```

11. Database / Storage Design

The project uses JSON files for lightweight local storage instead of a separate database server.

File	Fields
rooms.json	room_no, type, price, status
guests.json	guest_id, name, phone, email
bookings.json	booking_id, guest_id, guest_name, room_no, room_type, price_per_night, nights, status, booking_date

12. Design Decisions and Rationale

Python was selected because the project is intended to demonstrate programming fundamentals, modular design, file handling, validation and object-oriented programming.

JSON was selected for storage because it is lightweight, human-readable and does not require a separate database server.

The application is divided into modules so that each major responsibility can be modified independently.

13. Implementation Details

- Classes are used for room, guest and booking management.
- JSON files provide persistent local storage.
- Input validation functions prevent invalid numeric values and empty mandatory fields.
- Booking status changes from Booked to Checked-In and then Checked-Out.
- Checking out a guest changes the room status back to Available.

Billing formulas:

```
Room Charge = Price per Night × Number of Nights
Tax = 5% × (Room Charge + Food/Service Charges)
Total = Room Charge + Food/Service Charges + Tax
```

14. Testing Approach

The project includes test_hotel.py using Python's unittest framework. The tests cover room-charge calculation, tax calculation, total bill calculation and booking status transition.

```
python -m unittest test_hotel.py
```

15. Challenges Faced

- Designing separate modules for different hotel operations.
- Maintaining room status after booking and checkout.
- Validating user input.
- Keeping data persistent using JSON files.
- Connecting billing with booking information.

16. Learnings and Key Takeaways

- Python functions and classes
- Object-oriented programming
- Lists and dictionaries
- File handling
- JSON data storage
- Input validation
- Exception handling
- Modular programming
- Unit testing
- Basic software project organization

17. Future Enhancements

- GUI using Tkinter
- SQLite/MySQL database
- User login and roles
- Online booking
- Food-order module
- Customer feedback
- Advanced analytics
- Automated email/SMS notifications

18. Project Structure

```
Hotel-Management-System/
■■■ main.py
■■■ room_manager.py
■■■ guest_manager.py
■■■ booking_manager.py
■■■ billing_manager.py
■■■ database.py
■■■ validators.py
■■■ reports.py
■■■ test_hotel.py
■■■ data/
```

- ■■■ rooms.json
- ■■■ guests.json
- ■■■ bookings.json
- README.md
- statement.md

19. References

- Python official documentation
- VITyarthi – Build Your Own Project guidelines